

ארכיטקטורה ותכנון מערכת זמן אמת

לסימולציית מוקד חירום עירוני

מחייב הבנה עמוקה (RTOS) פיתוח מערכות משובצות מחשב מבוססות מערכות הפעלה לזמן אמת מסמך מחקר ותכנון זה מציג (IPC) של עקרונות תזמון, סנכרון, ניהול זיכרון ותקשורת בין תהליכים (City Emergency Dispatch Simulation) "ארכיטקטורה הנדסית מלאה לפיתוח מערכת "סימולציית מוקד חירום עירוני של חברת Nucleo-F756ZG המערכת מתוכננת לרוץ על גבי לוח הפיתוח (Dispatch Simulation). STM32CubeMX, הכולל ARM Cortex-M7 320 המבוסס על מעבד, STM32CubeMX סביבת הפיתוח מבוססת על ¹Flash ו-1 מגה-בייט של זיכרון SRAM קילובייט של זיכרון העטופה בשכבת התאימות FreeRTOS לייצור קוד האתחול, ומערכת ההפעלה CMSIS-OS v2⁴.

הדרישות הלוגיות של המערכת מתארות זרימת מידע דינמית: אירועי חירום נוצרים באקראי הנתב מנתב את הקריאות, קובע את דחיפותן ומעביר אותן (Dispatcher) ומועברים לנתב מרכזי למחלקות השונות (אמבולנס, משטרה, מכבי אש וקורונה)⁷. המחלקות מוגבלות במשאביהן, ולכן נדרש מנגנון מתקדם לניהול משאבים המאפשר "השאלה" של משאבים פנויים ממחלקות אחרות המנותבת החוצה דרך (Logging) בעת עומס. במקביל, כל אירוע חייב להיות מתועד במערכת רישום שאינו גוזל זמן מעבד יקר מליבת המערכת⁷ (Thread-Safe) באופן בטוח לתהליכים, UART-ממשק ה

מודל (Task Management) ארכיטקטורת המשימות

התזמון

Preemptive Priority-Based (מספקת תזמון מבוסס עדיפויות מונע FreeRTOS מערכת ההפעלה היא זו (Ready) "שבו המשימה בעלת העדיפות הגבוהה ביותר שנמצאת במצב "מוכנה, (Scheduling) שתקבל את זמן המעבד¹⁰. חלוקה נכונה של המערכת למשימות בלתי תלויות היא קריטית למניעת והבטחת תגובתיות בזמן אמת (Blocking) חסימות אשר osThreadNew משימות מוגדרות ונוצרות באמצעות קריאה לפונקציה CMSIS-OS v2 בסביבת ועבור (TCB) עבור בלוק הבקרה של המשימה (FreeRTOS (Heap-מקצה זיכרון דינמי מתוך ערימת ה שלה⁴. לחלופין, ניתן להשתמש בהקצאה סטטית אם מוגדר (Stack) המחסנית configSUPPORT_STATIC_ALLOCATION¹⁴.

שם המשימה (Task)	עדיפות (CMSIS-OS v2 Priority)	גודל מחסנית (Words) מומלץ	תיאור ותפקיד במערכת
Event Generator	osPriorityAboveNormal	256	מחולל הקריאות האקראיות. מדמה את האזרחים ומייצר אירועי חירום במרווחי זמן משתנים. מעביר את האירועים לתור

הנתב.			
משימת הניתוב המרכזית. שולפת אירועים, בוחנת את סוגם ואת רמת הדחיפות שלהם, ומנתבת אותם לתורי המחלקות הרלוונטיות.	512	osPriorityHigh	Dispatcher Task
ארבע משימות נפרדות (משטרה, כבאות, אמבולנס, קורונה). כל אחת ממתינה בתור שלה, מנהלת את המשאבים, ומבצעת "את" העבודה.	256	osPriorityNormal	Department Tasks (x4)
אחראית בלעדית על התקשורת מול USART3-חומרת ה מקבלת בקשות הדפסה מתור ייעודי ומשדרת אותן כדי DMA באמצעות לשחרר את המעבד.	512	osPriorityLow	UART Gatekeeper

כאשר משימת מחלקה (לדוגמה, אמבולנס) מטפלת באירוע, היא מדמה את זמן הטיפול. קריטי מבצעת המתנה פעילה HAL_Delay הפונקציה. HAL_Delay במקום ב osDelay להשתמש בפונקציה שחוסמת את המעבד לחלוטין ומונעת ממשימות בעדיפות נמוכה או שווה לרוץ⁹. לעומת (Busy-Wait) ומאפשרת למתזמן של (Blocked State) מעבירה את המשימה למצב חסום osDelay, זאת להעביר את השליטה למשימה הבאה בתור⁴ FreeRTOS.

HAL של ספריית ה (Timebase Source) כדי למנוע התנגשויות זמן, חובה לשנות את מקור התזמון FreeRTOS מערכת ההפעלה. TIM6 לטיימר חומרה ייעודי, כגון SysTick מ-STM32CubeMX בסביבת Context) לצורך ניהול זמני המערכת ומעבר בין הקשרים SysTick-דורשת שליטה בלעדית בפסיקת ה ושימוש משותף יוביל לאנומליות בתזמון ולתקיעות של המערכת⁴, (Switching).

וניתוב אירועים (IPC) מנגנוני תקשורת בין תהליכים

בטוחים ויעילים. IPC העברת המידע בין מחולל האירועים לנתב, ובין הנתב למחלקות, דורשת מנגנוני בשל סכנות של דריסת RTOS הגישה של שימוש במשתנים גלובליים נפסלת על הסף במערכות ¹⁸(Preemption) בעת קטיעת תהליכים (Race Conditions) ותנאי תחרות (Data Corruption) נתונים (Message Queues) הפתרון האופטימלי להעברת קריאות חירום במערכת הוא שימוש בתורי הודעות לתוך מרחב הזיכרון של התור, מה שמאפשר (Pass by Copy) תורי הודעות מעתיקים את הנתונים למשימה השולחת להמשיך להשתמש במשתנה המקורי מיד לאחר השליחה ללא חשש משינויים שיכלול את קוד המחלקה, מזהה (Struct) בלתי רצויים¹⁸. כל אירוע חירום ייוצג על ידי מבנה נתונים ייחודי, חותמת זמן ואת עדיפות האירוע.

תפקיד ויישום בארכיטקטורה	פונקציות ליבה	סוג נתונים	IPC מנגנון CMSIS-OS v2-ב
העברת אירועים ממחולל הקריאות לנתב, ומהנתב לכל אחת מארבע המחלקות. מאפשר ניהול תורים מסודר ומניעת אובדן מידע.	osMessageQueueNew, osMessageQueuePut, osMessageQueueGet	EmergencyEvent_t	osMessageQueueId_t
העברת מצביעי זיכרון למחרוזות הלוגים מכלל המשימות אל UART-משימת ה Gatekeeper.	osMessageQueuePut	char* (Pointer)	osMessageQueueId_t
הקצאה דינמית בטוחה ומהירה של בלוקי מחרוזות להדפסה, המונעת קיטוע זיכרון (Fragmentation).	osMemoryPoolNew, osMemoryPoolAlloc, osMemoryPoolFree	בלוקי זיכרון	osMemoryPoolId_t
הגנה על המבנה הגלובלי המייצג את סך המשאבים (אמבולנסים, ניידות) בעיר, המאפשר	osMutexNew, osMutexAcquire, osMutexRelease	משאב נעילה	osMutexId_t

			"השאלה" בטוחה של משאבים.
--	--	--	--------------------------

CMSIS-OS v2 ניהול תעדוף באירועי חירום באמצעות

הדרישה המערכתית מחייבת שאירועים קריטיים, כגון שריפות או קריאות רפואיות דחופות, יקבלו טהורה, תורים מתנהגים על פי עקרון FreeRTOS קדימות בטיפול לעומת אירועים שגרתיים⁷. בסביבת ללא יכולת מיון פנימית על בסיס עדיפות¹⁸. עם זאת, השימוש LIFO ראשון נכנס ראשון יוצא) או FIFO מספק פתרון אלגנטי ומובנה לאתגר זה CMSIS-OS v2 במעטפת msg_prio מקבלת פרמטר ייעודי בשם CMSIS-OS v2 בממשק osMessageQueuePut הפונקציה כאשר הנתב מעביר הודעה לתור של מחלקה, הוא מנתח את האירוע ומקצה לו ערך¹⁸ (uint8_t מסוג) מסדר באופן אוטומטי את ההודעות בתוך התור כך CMSIS עדיפות מספרי. תור ההודעות של שהודעות בעלות ערך עדיפות גבוה יותר יוצבו בראש התור וישלפו ראשונות על ידי פונקציית מנגנון זה מבטיח שאם מחלקת האמבולנסים עמוסה ובתור ממתנות¹⁸ osMessageQueueGet מספר קריאות, קריאה המוגדרת כהחייאה (עדיפות גבוהה) תעקוף באופן מיידי קריאות שגרתיות במבנה הנתונים הפנימי של התור ותטופל ברגע שמשאב יתפנה.

ניהול משאבי מערכת ומנגנון ההשאלה הבין-מחלקתי

כל מחלקה במערכת מנהלת מספר מוגבל של משאבים לצורך טיפול באירועים. על פי הדרישות, קיימים 4 אמבולנסים, 3 ניידות משטרה, 2 כבאיות ו-4 רכבי קורונה⁷. כאשר מחלקה ממצה את מכסת המשאבים שלה, עליה לחפש משאב פנוי ממחלקה אחרת ("השאלה"). במידה ואף משאב אינו פנוי, האירוע נשאר להמתנה בתור.

(Counting) הגישה הבסיסית ביותר לניהול מספר משאבים זהים היא שימוש בסמפור מנייה אשר מאותחל למספר המשאבים הפנויים⁴. אולם, מנגנון ההשאלה מורכב בהרבה: אם, (Semaphore משימת מחלקת הרפואה ניסתה לקחת סמפור של מחלקת המשטרה ונכשלה, או לקחה אותו אך בו שתי מחלקות (Deadlock) צריכה להחזיר אותו מאוחר יותר, נוצרת סכנה מוחשית למצב של קיפאון (Ownership) ממתנות לנצח לסמפורים אחת של השנייה¹⁰. סמפורים אינם מצמידים מזהה בעלות ולכן כל משימה יכולה לשנותם, דבר המקשה על מעקב לוגי מדויק אחר המשאבים ה"מושאלים"¹⁰. הפתרון האדריכלי הנכון לדרישה זו הוא יצירת אובייקט נתונים מרכזי וגלובלי (לדוגמה, מבנה נתונים המכיל ארבעה מונים המייצגים את המלאי הזמין של כל מחלקה), והגנה עליו באמצעות קריאה⁴. Mutex בעזרת (Mutual Exclusion) הדדית בלעדית.

(Priority Inheritance) מנגנון הורשת עדיפויות

יש תכונה מובנית קריטית למערכות זמן אמת: FreeRTOS ב-Mutex- בשונה מסמפור בינארי, ל מתרחשת כאשר (Priority Inversion) תופעת היפוך עדיפויות. (Priority Inheritance) הורשת עדיפויות ובכך מונעת ממשימה בעדיפות גבוהה הזקוקה לאותו Mutex- משימה בעדיפות נמוכה נועלת את ה מלהמשיך לרוץ. אם משימה בעדיפות בינונית קוטעת את המשימה בעדיפות הנמוכה, Mutex פותר זאת בכך שברגע Mutex- המשימה בעדיפות הגבוהה חווה עיכוב שאינו חסום בזמן¹⁰. ה שהמשימה בעדיפות הגבוהה דורשת את המיוטקס התפוס, מערכת ההפעלה מקפידה זמנית את רמת העדיפות של המשימה בעדיפות הנמוכה (המחזיקה במיוטקס) לרמת העדיפות של המשימה

הגבוהה. הדבר מבטיח שהמשימה הנמוכה תסיים את עבודתה במהירות האפשרית, תשחרר את ותאפשר למערכת לחזור לתפקוד תקין¹⁰, Mutex-ה.

אלגוריתם הטיפול של מחלקה:

1. osMessageQueueGet המשימה שולפת הודעה מתור המחלקה באמצעות.
2. osMutexAcquire המשימה מבקשת לנעול את המיוטקס הגלובלי באמצעות.
3. לאחר קבלת הנעילה, היא בודקת את המונה המקומי שלה. אם הוא גדול מאפס, היא מפחיתה אותו באחד.
4. אם המונה שלה הוא אפס, היא סורקת את המונים של המחלקות האחרות. אם נמצא מונה שגדול מאפס, היא מפחיתה אותו ורושמת לעצמה ממך "השאלה" את המשאב⁷.
5. אם כלל המונים בעיר עומדים על אפס, המשימה משחררת את המיוטקס באמצעות osMutexRelease כדי לנסות (Timeouts) ועושה שימוש בפונקציה משהה עם זמני פסק-זמן osMutexRelease שוב, או מעבירה את האירוע לטיפול של מנגנון ניהול התקלות במידה ומשאב אותה, המיוטקס משוחרר. המשימה מעכבת את עצמה כדי לדמות עבודה ובסיום שוב נועלת את המיוטקס, מחזירה את המשאב (למחלקה המקורית ממנה, osDelay) נלקח), ומשחררת את המיוטקס.

UART Gatekeeper רישום אירועים בטוח באמצעות

I-DMA

בסביבת ריבוי תהליכים מהווה נקודת כשל (UART) הוצאת נתונים דרך יציאת התקשורת הטורית במערכת משובצת פוגע בביצועים. המרת printf נפוצה. שימוש ישיר בפונקציות מסורתיות כמו דורשת זמן עיבוד רב, ושידור (snprintf או sprintf טיפוסים) כגון מספרים למחרוזות באמצעות של 115200 חוסם את המעבד למשך מילישניות ארוכות לכל (Baud Rate) מחרוזות בבאוד-רייט הודעה²⁴. אם משימת מחלקה תעצור כדי לשדר מחרוזת פלט, היא לא תוכל לקלוט ולעבד את של HAL-האירועים הבאים בזמן סביר. בנוסף, אם שתי משימות קוראות במקביל לפונקציות הסטנדרטיות גם אין C-התקשורת תושחת ונתונים ידרסו⁹. קריאות למנגנוני ספריות ה-UART-ה בהגדרות Use newlib reentrant-מטבען ודורשות הפעלה של אפשרות ה (Thread-Safe) בטוחות הנגרמים מהשחתת נתונים פנימיים של פונקציות HardFaults מערכת ההפעלה על מנת למנוע הספרייה⁹.

בספרות התחום Best Practice-הארכיטקטורה הנכונה ביותר לפרויקט זה, אשר נתמכת גם כ בשילוב מאגרי זיכרון, **UART Gatekeeper Task** מורכבת ממימוש תבנית FreeRTOS ובתיעוד של מקבלת בעלות בלעדית Gatekeeper-משימת ה⁸ (DMA) וגישה ישירה לזיכרון (Memory Pools) אף משימה אחרת אינה רשאית לגשת ישירות לרכיבי החומרה של UART⁸-ומוחלטת על פריפריאל ה-UART.

Gatekeeper שלבי הרישום וההדפסה בארכיטקטורת

הבאים IPC-כדי להשיג דטרמיניזם ומהירות ביצוע אופטימליים, נדרש שילוב של מנגנוני ה

1. משימות המעוניינות להדפיס פלט זקוקות לזיכרון עבור: **(Memory Pool) הקצאת מאגר זיכרון** היא פעולה איטית, יוצרת קיטוע בזיכרון (malloc) המחרוזת. הקצאת זיכרון דינמית קלאסית CMSIS-OS v2 ואינה תמיד מתאימה למערכות זמן אמת עמוסות⁴. סביבת (Fragmentation) בעת אתחול המערכת, יוגדר מאגר של עשרות⁴ osMemoryPool-מספקת את אובייקט ה בלוקים בגודל קבוע (למשל, 80 בתים כל אחד)³¹.

2. כאשר משימת מחלקה מסיימת טיפול באירוע, היא משיגה בלוק: **(Producer) יצירת ההודעה**. המשימה מעצבת osMemoryPoolAlloc מהמאגר בתוחלת זמן אפסית באמצעות הפונקציה `snprintf`²⁵. את המחרוזת לתוך הבלוק הזה באמצעות
3. **שליחת ההודעה דרך תור**: המשימה לא שולחת את המחרוזת כולה. במקום זאת, היא שולחת דרך Gatekeeper של אותו בלוק אל תור ההודעות הייעודי של ה **(Pointer)** רק את המצביע העברת מצביע של 4 בתים הינה פעולה כמעט מיידית, `osMessageQueuePut`⁴. הפונקציה המשימה חוזרת לפעולתה השוטפת.
4. ממתינה בתור ההודעות UART Gatekeeper-משימת ה: **DMA (Consumer) שידור באמצעות** Gatekeeper-ברגע שמתקבל מצביע, ה `osMessageQueueGet` למצביעים חדשים בעזרת עם המצביע HAL של ספריית ה `HAL_UART_Transmit_DMA` מפעיל את הפונקציה המובנית שואב את הנתונים מהזיכרון (DMA (Direct Memory Access) לאותה מחרוזת וגודלה²⁷. ה לאחר (CPU) מבלי לערב את יחידת העיבוד המרכזית, UART ומזרים אותם באופן עצמאי אל ה מתן הפקודה¹⁵.
5. Gatekeeper-עובד, על משימת ה DMA-בעוד ה: **(ISR Synchronization) סנכרון מול הפסיקה** DMA-בעוד ה HAL-ינסה להעביר הודעה נוספת מהתור אל ה Gatekeeper-להמתין. אם ה וההודעה תאבד. לכן, מיד לאחר הפעלת (`HAL_BUSY`) עובד, הספרייה תחזיר קוד שגיאה על סמפור בינארי ומעביר את עצמו `osSemaphoreAcquire`-קורא ל Gatekeeper ה-DMA, למצב חסום¹⁵. זה מוודא שהמשימה לא תמשוך מצביעים נוספים.
6. מסיים להעביר את התו האחרון DMA-השלמת השידור ושחרור הבלוק: כאשר בקר ה HAL: של ה Callback-במחרוזת, הוא מעורר פסיקת חומרה, אשר מובילה להפעלת פונקציית ה בתוך פונקציית פסיקה זו, אסור לבצע פעולות חוסמות. הפסיקה `HAL_UART_TxCpltCallback` המנהלת `xSemaphoreGiveFromISR` או לשיטה המקבילה (`osSemaphoreRelease`-קוראת ל ממצב החסימה Gatekeeper-כדי לשחרר את ה (`pxHigherPriorityTaskWoken` את המשתנה שקם לחיים משחרר את בלוק הזיכרון Gatekeeper-וליידיע אותו שהשידור הסתיים¹⁴. לבסוף, ה וחוזר להמתין למצביע הבא בתור¹⁶, `osMemoryPoolFree` חזרה למאגר באמצעות

סביבת ביצוע	API מנגנון / פונקציית CMSIS-OS v2-ב	שלב בזרימת הנתונים של Gatekeeper-ה
משימות מחלקות (Producer)	<code>osMemoryPoolAlloc</code>	השגת בלוק זיכרון לרישום
משימות מחלקות	<code>snprintf</code>	עיצוב נתוני הלוג לבלוק
משימות מחלקות	<code>osMessageQueuePut</code>	העברת המצביע ללוג
UART Gatekeeper משימת	<code>osMessageQueueGet(..., osWaitForever)</code>	קבלת מצביע לשידור
UART Gatekeeper משימת	<code>HAL_UART_Transmit_DMA</code>	אתחול תהליך שידור חומרה

חסימה עד לסיום השידור	osSemaphoreAcquire (בינארי)	UART Gatekeeper משימת
טריגר סיום פעולת חומרה	HAL_UART_TxCpltCallback	(ISR) פסיקת חומרה
שחרור המשימה החסומה	osSemaphoreRelease	(ISR) פסיקת חומרה
שחרור המשאב התורן	osMemoryPoolFree	UART Gatekeeper משימת

שיטה זו מבטיחה כי הדפסת נתונים לא תפגע בביצועי הסימולציה. כל המשימות מדפיסות באופן עקיף, ואילו המעבד פנוי לחלוטין לקבל אירועים חדשים בזמן שהחומרה ההיקפית מבצעת את פעולות הקלט/פלט הארוכות¹⁵.

STM32F756ZG Nucleo-144: תצורת חומרה ספציפית

מצוידים ברכיב STM32F756ZG-המאכלסים בקרים כדוגמת ה ST של חברת Nucleo-144-לוחות ה (Virtual המספק, מעבר ליכולות צריבה ודיבאג, גם מממשק חיבור סיריאלי וירטואלי ST-LINK/V2-1 COM Port) שדרכו מוצגים הלוגים למחשב האישי¹.
פיני התקשורת הדיפולטיביים: STM32CubeMX קיימת אנומליה ידועה בעת הגדרת הלוח בתוכנת אינם מקונפגים נכון בחלק מגרסאות מחולל הקוד²⁸. בעוד ST-LINK-שמחובר ל UART-לממשק ה PB10 ו-PB11 קונפיגורציית הפינים עשויה להצביע על, USB-הוא זה המחובר לממשק ה USART3-שה אל USART3-באופן ידני לחיבורי ה (Remapping) שהם חיבורים צדדיים. על המפתח לבצע שינוי תפקיד בממשק התוכנה¹. כשלון בשינוי זה יוביל ללוגים הנשלחים לחלל ריק (TX) PD9- ו (RX) PD8 הפינים בתוכנת המחשב (Terminal) מבלי שיופיעו במסוף.

תפקיד תקשורת	רכיב פנימי (Peripheral)	פין במעבד (Pin) Nucleo-F756ZG-ב	הפין הווירטואלי (VCP) המשותף
(TX) שידור נתונים	USART3	PD8	ST-LINK RX
(RX) קליטת נתונים	USART3	PD9	ST-LINK TX

מכוון לתדר יציב USART3-כי נתב התדרים הפנימי של ה Clock-הערה: יש לאמת בהגדרות מחולל ה (APB או דרך מחלק MHz)(למשל, שעון הליבה 216).

ניהול פסיקות, פערים בעדיפויות וטיפול בתקלות זיכרון

כדי לייצר מערכת שעומדת במבחני המציאות ולא קורסת במצבי לחץ פתאומיים (כגון שטף אירועים הנובע מתאונת שרשרת, במונחי הסימולציה), יש ליישם תכנון הנדסי למצבי קיצון.

1. RTOS-מול ה (Interrupt Priorities) קונפיגורציית עדיפויות הפסיקות

מתערכת באופן ישיר ביכולות בקר ARM Cortex-M עבור מעבדי סדרת FreeRTOS מערכת ההפעלה FreeRTOSConfig.h של המעבד⁴. בסביבת התכנות, קיים ערך מפתח ב (NVIC) הפסיקות המקונון אשר פעמים רבות מקבל את ערך (configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY בשם ברירת המחדל 5)⁴.

חייבת להיות RTOS-ערך זה מהווה חוצץ קריטי: כל פסיקת חומרה אשר מיועדת לתקשר עם ה משמעות הדבר היא ערך, ARM מקונפגט בעדיפות פסיקה זהה או נמוכה יותר מערך זה (במונחי מספרי של 5 ומעלה, היות ו-0 היא הפסיקה בעלת העדיפות הגבוהה ביותר)⁴. בארכיטקטורה המטפלת בתהליך השידור של DMA-וכן פסיקה ה, UART-המוצעת כאן, הפסיקה של ה שאינה גבוהה מ-5. במידה (Preemption Priority) חייבות להיות מקונפגט בעדיפות, Gatekeeper-ה ליבת מערכת, osSemaphoreRelease-תוגדר בעדיפות 0, ותפעיל מתוכה קריאה ל DMA-ופסיקה ה configASSERT¹⁴.-ההפעלה תזהה חריגה ותכניס את כלל המערכת ללולאה אין סופית בפונקציית ה RTOS I-DMA¹⁵. זהו כשל נפוץ מאוד בתהליכי פיתוח המשלבים

2. (Stack Overflow Detection) איתור וזיהוי של גלישת מחסנית

ומרכיבות מחרוזות מורכבות, עושות שימוש sprintf המשימות במערכת המשתמשות בפונקציית שמוקצית עם בניית התהליך²⁴. (Task Stack) אינטנסיבי במחסנית הזיכרון המקומית של המשימה כברירת מחדל, גודל המחסנית עשוי לעמוד על 128 מילים בלבד (512 בתים במעבד בעל ארכיטקטורת 32-ביט)⁴. פונקציית העיצוב הסטנדרטית עלולה למשוך למעלה מ-150 בתים באופן HardFaults²⁴-פנימי²⁴, מה שיביא לדריסת זיכרון של משימות אחרות ולגרום לאפקטים בלתי צפויים ול לערך FreeRTOSConfig.h-כחלק מניהול התקלות בארכיטקטורה, יש לשנות את ההגדרה ב הגדרה זו גורמת לליבה להטביע חתימת זיכרון. configCHECK_FOR_STACK_OVERFLOW = 2. בקצוות המחסנית של כל משימה בעת יצירתה. במהלך החלפת הקשר (x5xשל 0 Pattern, "תבנית הליבה סורקת את הזיכרון בקצוות. אם הערך שונה, הדבר מצביע בוודאות על (Context Switch) בשם (Hook Function) הליבה תזנק באופן מיידי לפונקציית הקרס. (Overflow) דריסה שבה יהיה ניתן לעצור את התוכנה לחלוטין ולספק משוב דיבאג vApplicationStackOverflowHook ברור למפתח תוך ציון שם המשימה שגרמה לגלישה⁴. שילוב טכניקות מניעה אלה מבסס קוד חסין שאינו קורס שתיקות במקרי קצה (Robust).

3. ניהול כשל ברמת משימות הסימולציה

עמידות המערכת מחייבת מענה גם ברמה הלוגית של הסימולציה⁷. נתב הקריאות המרכזי או מחולל האירועים מעבירים הודעות לתורים. תורי ההודעות מוקצים בגודל מוגבל (Dispatcher Task) של מקרי חירום, התורים יתמלאו (Burst) (למשל, 10 מקומות לתור האמבולנס)¹⁸. בזמן התפרצות במהירות אם צוותי הרפואה עדיין עסוקים.

שימוש מועיל בפרמטר Timeout¹⁸ מועבר פרמטר של פסק-זמן osMessageQueuePut לפונקציית אלא יקבע, osWaitForever לא יקרא לפונקציה עם המזהה Dispatcher-זה ימנע עצירת המערכת. ה גבול של, למשל, 50 מילישניות המתנה לכל היותר¹⁸. אם הפונקציה תחזיר את קוד השגיאה למקרה בו מוגדר 0 כזמן המתנה, osErrorResource בגרסאות המוקדמות גם) osErrorTimeout לבצע Dispatcher-שהתור מלא כליל⁴. כתגובה למקרה הקצה הזה, על ה Dispatcher-הדבר יסמן ל הבאות: לשלוח את האירוע לתור המתנה פנימי שלו לחזרה Fault Management-אחת מפעולות ה

בשלב מאוחר יותר, להמיר את דחיפות האירוע, או במקרים גרועים ביותר לדלג עליו תוך רישום הערה יכולת השליטה UART Gatekeeper-קריטית בקונסולה באמצעות שליחת הודעת אזהרה למשימת הדינמית הזו, המבוססת על שימוש בקוד שגיאות החזר במקום הנחות א-פריריות, היא ליבה של ארכיטקטורת זמן-אמת נכונה⁷.

עבודות שצוטטו

1. Nucleo F756ZG - Zephyr Documentation, https://docs.zephyrproject.org/latest/boards/st/nucleo_f756zg/doc/index.html
2. STMicroelectronics NUCLEO-F756ZG - Keil.arm.com, <https://www.keil.arm.com/boards/stmicroelectronics-nucleo-f756zg-revb-2251244/guide/>
3. NUCLEO-F756ZG | Product - STMicroelectronics, <https://www.st.com/en/evaluation-tools/nucleo-f756zg.html>
4. FreeRTOS_v2.0 - CMSIS OS API.pdf
5. cmsis-rtx - Arm Keil, <https://www.keil.arm.com/packs/cmsis-rtx-arm/versions/>
6. CMSIS_5/CMSIS/DoxyGen/RTOS2/src/cmsis_os2.txt at develop - GitHub, https://github.com/ARM-software/CMSIS_5/blob/develop/CMSIS/DoxyGen/RTOS2/src/cmsis_os2.txt
7. Final Project- City Emergency Dispatch Simulation.pdf
8. [RTOS/multithreaded] one task/thread per device... effective alternatives? : r/embedded - Reddit, https://www.reddit.com/r/embedded/comments/9tm3ef/rtosmultithreaded_one_thread_per_device/
9. STM32 FreeRTOS: CMSIS-RTOS V2 Setup, Tasks & LED Blink - ControllersTech, <https://controllerstech.com/stm32-freertos-tutorial-cubemx-led-example/>
10. __Real-Time_Embedded_Concepts_October-2023.pdf
11. STM32 FreeRTOS: Multiple Tasks, Priorities & Preemption - ControllersTech, <https://controllerstech.com/stm32-freertos-multiple-tasks-priorities-preemption/>
12. Why Your STM32 Projects Need FreeRTOS (And How to Set It Up in 10 Minutes) - Medium, <https://medium.com/@arunrawat2112/why-your-stm32-projects-need-freertos-and-how-to-set-it-up-in-10-minutes-1f77f4b91eaf>
13. CMSIS-RTOS2 Overview and Setup Guide | PDF | Scheduling (Computing) - Scribd, <https://www.scribd.com/document/902036949/CMSIS-RTOSv2>
14. FreeRTOS_Reference_Manual_V10.0.0.pdf
15. OmurCeran/Autosar_Bsw_Mini: AUTOSAR Classic BSW concepts (COM, DEM, SchM, RTE) implemented from scratch on STM32F4 + FreeRTOS. Demonstrates EPS race condition and its implicit RTE solution. · GitHub, https://github.com/OmurCeran/Autosar_Bsw_Mini
16. CMSIS-FreeRTOS/CMSIS/RTOS2/FreeRTOS/Source/cmsis_os2.c at main - GitHub, https://github.com/ARM-software/CMSIS-FreeRTOS/blob/main/CMSIS/RTOS2/FreeRTOS/Source/cmsis_os2.c
17. STMicroelectronics STM32F103 ARM Cortex-M3 Demo - FreeRTOS™, <https://www.freertos.org/Documentation/02-Kernel/03-Supported-devices/04-D>

- [emos/ST-Microelectronics/portstm32iar](https://www.st.com/en/microelectronics/portstm32iar)
18. STM32 FreeRTOS Queue Tutorial: Inter-Task Communication with CMSIS-OS V2 - ControllersTech,
<https://controllerstech.com/stm32-freertos-queue-inter-task-communication/>
 19. FreeRTOS queues,
<https://freertos.org/Documentation/02-Kernel/02-Kernel-features/02-Queues-mutexes-and-semaphores/01-Queues>
 20. STM32 FreeRTOS Event Flags: osFlagsWaitAll, WaitAny & ISR Callbacks - ControllersTech,
<https://controllerstech.com/stm32-freertos-event-flags/>
 21. STM32-Cube IDE-FreeRTOS Intertask Communication Part - 1 - Kalapi Infotech,
<https://kalapiinfotech.in/stm32-cube-ide-freertos-intertask-communication-part-1/>
 22. CMSIS-RTOS2: Message Queue - GitHub Pages,
https://arm-software.github.io/CMSIS_6/main/RTOS2/group_CMSIS_RTOS_Message.html
 23. Priority inheritance on mutexes not working any more in freeRTOS? | Community,
<https://community.st.com/stm32-mcus-embedded-software-32/priority-inheritance-on-mutexes-not-working-any-more-in-freertos-157210>
 24. STM32 FreeRTOS Stack Management: Monitor Usage & Detect Overflow - ControllersTech,
<https://controllerstech.com/stm32-freertos-stack-management-monitor-detect-overflow/>
 25. Printf Debugging - Handy Hint #4 - STMicroelectronics Community,
<https://community.st.com/others-stm32-mcus-related-142/printf-debugging-handy-hint-4-151472>
 26. CubeMX HAL - Is Thread Safe? - STMicroelectronics Community,
<https://community.st.com/stm32-mcus-embedded-software-32/cubemx-hal-is-thread-safe-67882>
 27. [SOLVED] FreeRTOS mutex issue in ISR with stm32 HAL and UART DMA TX | Community,
<https://community.st.com/stm32-mcus-embedded-software-32/solved-freertos-mutex-issue-in-isr-with-stm32-hal-and-uart-dma-tx-24495>
 28. UART and Ethernet on the STM32 Nucleo-F756ZG - LinuxJedi's /dev/null,
<https://linuxjedi.co.uk/uart-and-ethernet-on-the-stm32-nucleo-f756zg/>
 29. Gatekeeper Tasks - ECE353: Introduction to Microprocessor Systems,
<https://ece353.engr.wisc.edu/freertos/gatekeeper-tasks/>
 30. Freertos - v2.0 - Cmsis Os API | PDF - Scribd,
<https://www.scribd.com/document/1009214509/Freertos-v2-0-Cmsis-Os-API>
 31. No implementation for CMSIS-RTOSv2 Memory Pools From CubeMX | Community,
<https://community.st.com/stm32-mcus-products-25/no-implementation-for-cmsis-rtosv2-memory-pools-from-cubemx-120279>
 32. Embedded C Notes | PDF | Pointer (Computer Programming) - Scribd,
<https://www.scribd.com/document/992304668/Embedded-C-Notes>
 33. printf to serial through a queue - STMicroelectronics Community,

- <https://community.st.com/stm32-mcus-products-25/printf-to-serial-through-a-q-queue-92035>
34. STM32 HAL_UART_Transmit_DMA issues - Kernel - FreeRTOS Community Forums, <https://forums.freertos.org/t/stm32-hal-uart-transmit-dma-issues/11714>
 35. UART TX DMA issues - STMicroelectronics Community, <https://community.st.com/stm32-mcus-products-25/uart-tx-dma-issues-69124>
 36. STM32 HAL_UART_Transmit_DMA issues - #7 by jianxuan - Kernel - FreeRTOS Forum, <https://forums.freertos.org/t/stm32-hal-uart-transmit-dma-issues/11714/7>
 37. Wrong USART3 pins on Nucleo-F756ZG - STMicroelectronics Community, <https://community.st.com/stm32cubemx-mcus-29/wrong-usart3-pins-on-nucleo-f756zg-7735>
 38. CMSIS_OS V2/FreeRTOS HardFault - Stack Overflow, <https://stackoverflow.com/questions/76104380/cmsis-os-v2-freertos-hardfault>
 39. UART Interrupts in FreeRTOS with STM32 HAL driver - Stack Overflow, <https://stackoverflow.com/questions/55945565/uart-interrupts-in-freertos-with-stm32-hal-driver>
 40. stm32f4/CMSIS/RTOS2/Template/cmsis_os.h at master - GitHub, https://github.com/adafruit/stm32f4/blob/master/CMSIS/RTOS2/Template/cmsis_os.h